CHAPTER 4 - SYSTEM DESIGN & DEVELOPMENT

4.1 Introduction

This chapter shows how the requirements in Chapter 3 were turned into a design and a working application. It presents the system architecture, the UML diagrams (use case, class, activity and sequence), the database design, the user interface, the development environment, the implementation of the key modules, and the version-control evidence from the team's GitHub repository.

TrackWIL replaces the PDF and email logbook with a web application. Students log workplace hours and evidence, the workplace mentor verifies each entry through a one-time link without needing an account, and the WIL coordinator and lecturers can see who is on track, who is waiting on a mentor and who is overdue. The source code is in the folder 05_Source_Code/trackwil of the repository.

4.2 System Architecture

TrackWIL is a single Next.js application that runs in one Node.js process. The browser communicates with this process over HTTP (HTTPS when hosted). The process serves the React pages, runs the business rules in Server Actions, and reads and writes the SQLite database. Uploaded evidence files are stored on the server's disk. Figure 1 shows the architecture.

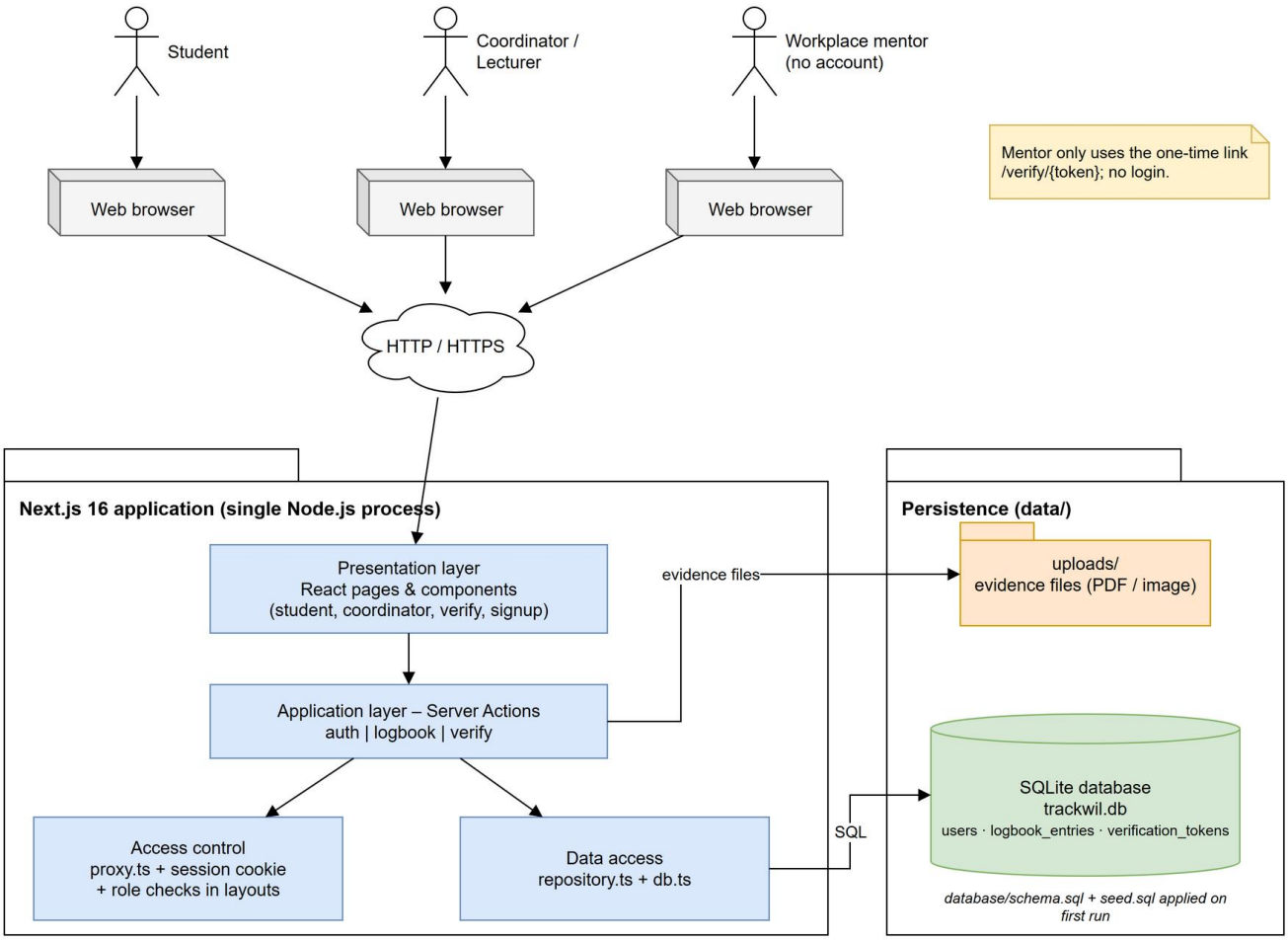


Figure 1: TrackWIL system architecture

The architecture has three logical layers. The presentation layer contains the pages and components each user sees. The application layer contains the Server Actions that validate input, check the user's role and apply the business rules. The data layer contains the repository and database modules; the repository is the only code that runs SQL, so the rest of the application never depends on how the data is stored. This separation is what allowed the team to replace the first prototype's JSON files with SQLite by changing only the data layer.

Table 1: Main components and their responsibilities

Component	Location in code	Responsibility	Requirements
Sign-in and sign-up pages	src/app/page.tsx, src/app/signup	Collect credentials; show validation errors	FR-01, FR-02
Student area	src/app/student	Logbook, new entry form, entry details, verification link	FR-04 to FR-06, FR-09, FR-10
Mentor verification page	src/app/verify/[token]	Show the entry and Approve / Reject buttons without sign-in	FR-07, FR-08
Coordinator dashboard	src/app/coordinator	Table of all students with status and hours (also used by lecturers)	FR-11
Server Actions	src/app/actions (auth, logbook, verify)	Validation, role checks and business rules	FR-01 to FR-08, FR-14
Access control	src/proxy.ts, layouts, src/lib/session.ts	Redirect signed-out users; enforce roles	FR-03
Evidence route	src/app/api/evidence/[fileName]	Serve evidence only to the owner and staff	FR-13
Repository and database	src/lib/repository.ts, src/lib/db.ts	All SQL queries and transactions; schema and seed on first run	FR-12, NFR-04
Metrics	src/lib/logbook-metrics.ts	Verified hours and On track / Awaiting / Overdue status	FR-10, FR-11

Data flow example (submitting an entry). The student clicks Submit for verification. The browser calls the submitEntryForVerificationAction Server Action, which loads the signed-in user, checks that the student owns the entry and that it is a Draft or Rejected entry with a mentor email, creates a random token and asks the repository to save the token and change the entry's status in one transaction. The page then shows the new status and the link.

Technology stack justification. Table 2 summarises the technologies chosen and why. The comparison of alternatives is discussed in Chapter 2, Section 2.4.

Table 2: Technology stack and justification

Layer	Technology	Why it was chosen
Front end	React 19 with Tailwind CSS 4	Component-based pages; utility classes give a consistent, responsive layout quickly
Application	Next.js 16 (App Router, Server Actions) with TypeScript 5	One project and one language for pages and server logic; type checking catches errors early
Database	SQLite through Node.js's built-in node:sqlite module	Real relational database with keys and constraints, but no server to install; one file that is easy to reset for demonstrations
Security	Node.js crypto (scrypt, randomBytes)	Standard, well-tested functions for password hashing and unguessable tokens; no extra dependencies
Version control	Git and GitHub	Branches, pull requests and a full history of each member's work

4.3 UML Design

Four UML diagrams describe the system from different views: the use case diagram (what each user can do), the class diagram (the main data types and code modules), the activity diagram (the end-to-end workflow) and the sequence diagram (the messages exchanged during mentor verification). The diagrams were drawn in draw.io, and their source files are in 04_System_Design_and_Development/diagrams.

Use Case Diagram

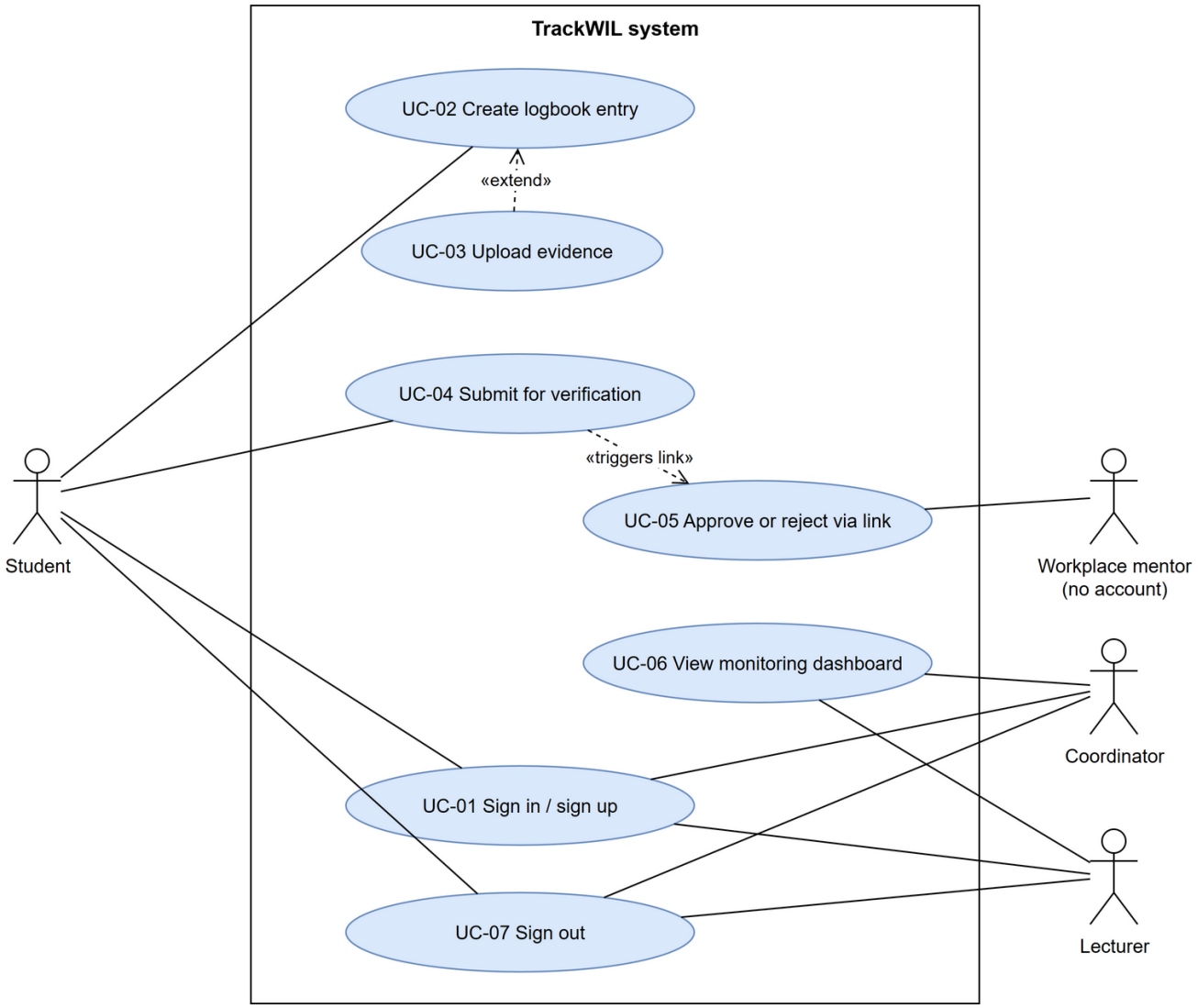


Figure 2: Use case diagram

Figure 2 shows four actors. The student signs in or signs up (UC-01), creates logbook entries (UC-02), optionally uploads evidence (UC-03, which extends UC-02 because it is optional), submits entries for verification (UC-04) and signs out (UC-07). Submitting an entry triggers the link that the workplace mentor uses to approve or reject the entry (UC-05); the mentor has no account and takes part in no other use case. The coordinator and the lecturer both sign in, view the monitoring dashboard (UC-06) and sign out. Each use case maps to the functional requirements listed in the SRS, Section 5.

Class Diagram

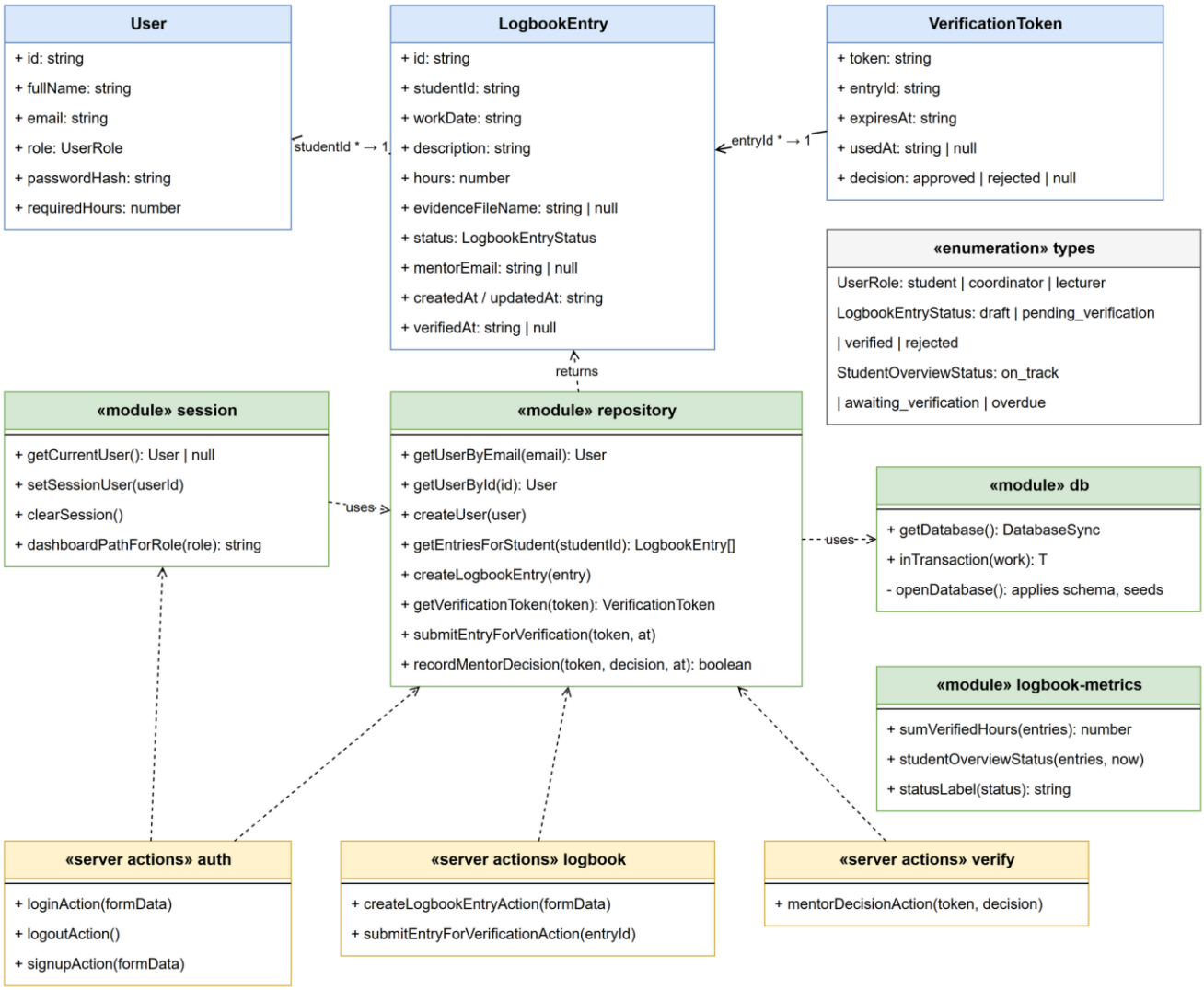


Figure 3: Class diagram of the main types and modules

Figure 3 shows the three domain types that are stored in the database (User, LogbookEntry and VerificationToken) and the enumerations they use. A LogbookEntry belongs to one User through studentId, and a VerificationToken belongs to one LogbookEntry through entryId. The lower half shows how the code is organised: the Server Actions (auth, logbook and verify) depend on the session and repository modules, the repository depends on the db module, and the logbook-metrics module calculates verified hours and the dashboard status from a student's entries. Because Next.js code is organised in modules and functions rather than classes, modules are shown with the «module» and «server actions» stereotypes.

Activity Diagram

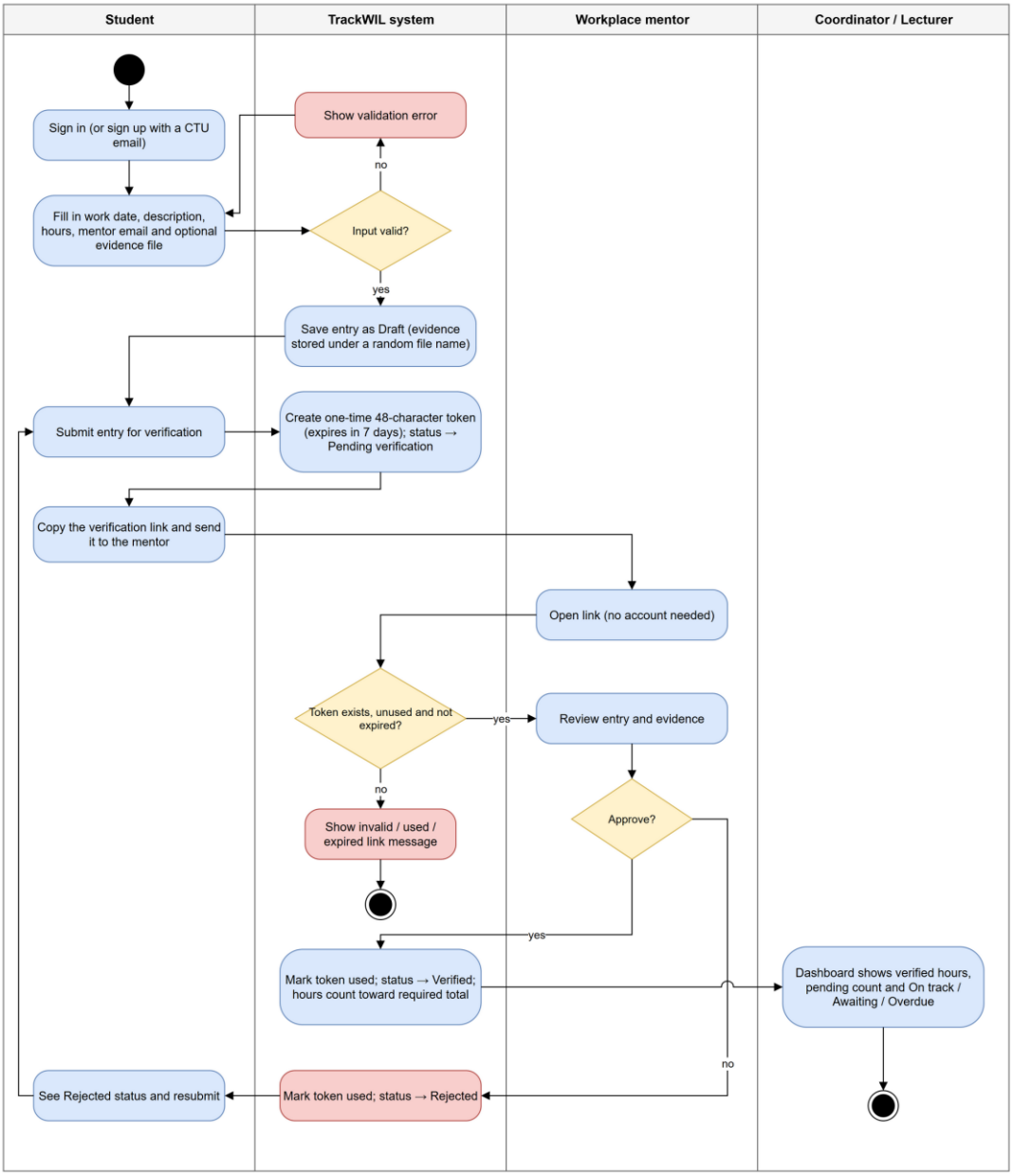


Figure 4: Activity diagram of the logbook and verification workflow

Figure 4 follows one logbook entry from creation to verification across four swimlanes. Invalid input returns the student to the form. A valid entry is saved as a Draft, then submitted, which creates a token and sets the status to Pending verification. When the mentor opens the link the system checks that the token exists, is unused and has not expired; if not, an error is shown and nothing changes. An approval sets the entry to Verified and adds its hours to the student's total on the dashboard. A rejection sets it to Rejected, and the student can resubmit it, which creates a new token.

Sequence Diagram

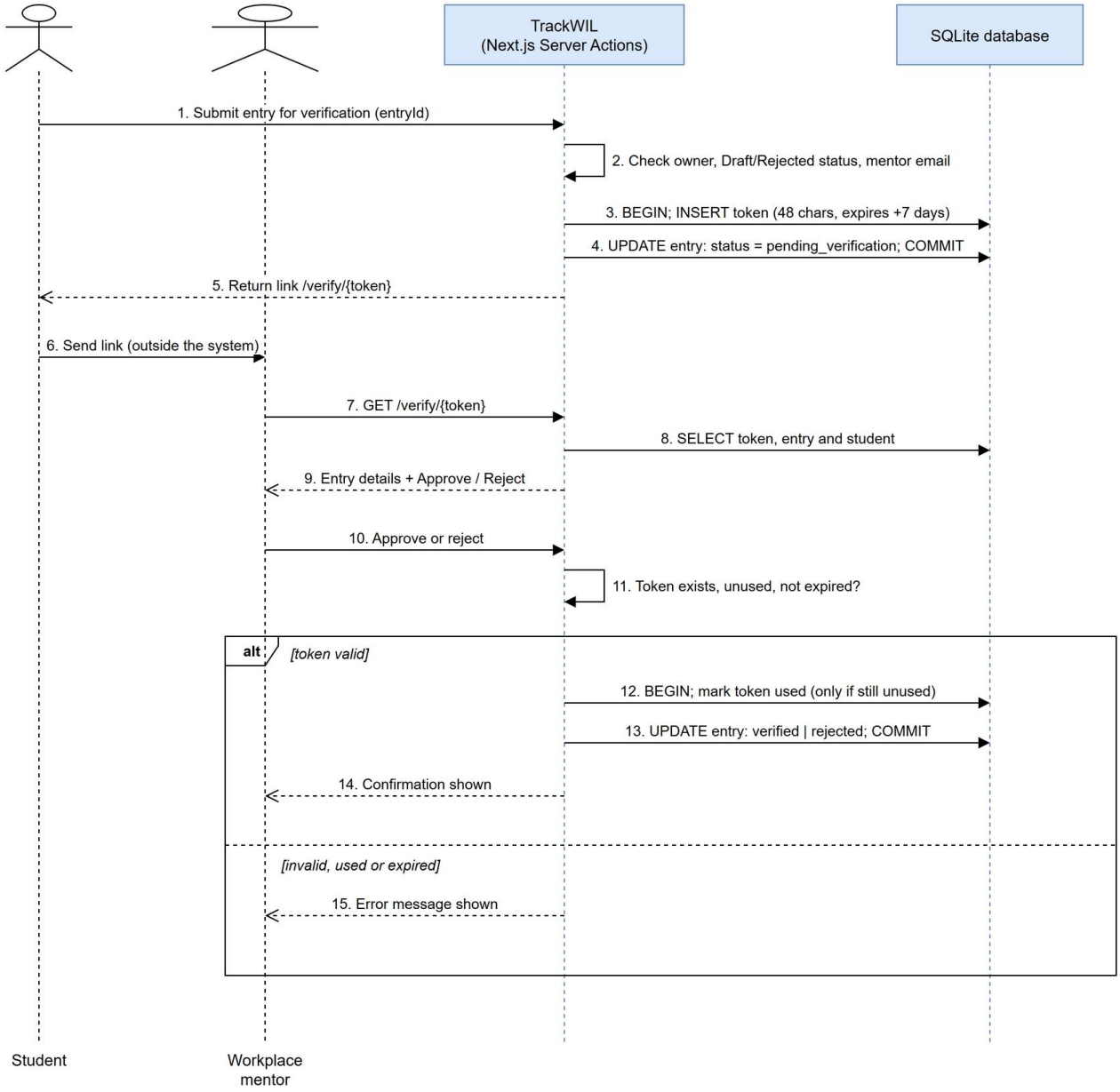


Figure 5: Sequence diagram of mentor verification

Figure 5 shows the messages between the student, the mentor, the TrackWIL server and the SQLite database during verification. Steps 3–4 and 12–13 each run inside a database transaction, so a token is never saved without the matching status change. Step 12 updates the token only if it is still unused, which prevents two decisions being recorded if the mentor clicks twice or opens the link in two tabs. The alt frame shows the two outcomes of the token check.

4.4 Database Design

TrackWIL uses an SQLite relational database with three tables. The schema is in 05_Source_Code/trackwil/database/schema.sql and the synthetic seed data is in seed.sql; both are applied automatically the first time the application starts. Figure 6 shows the entity-relationship diagram, and the full data dictionary is in the Database Design document.

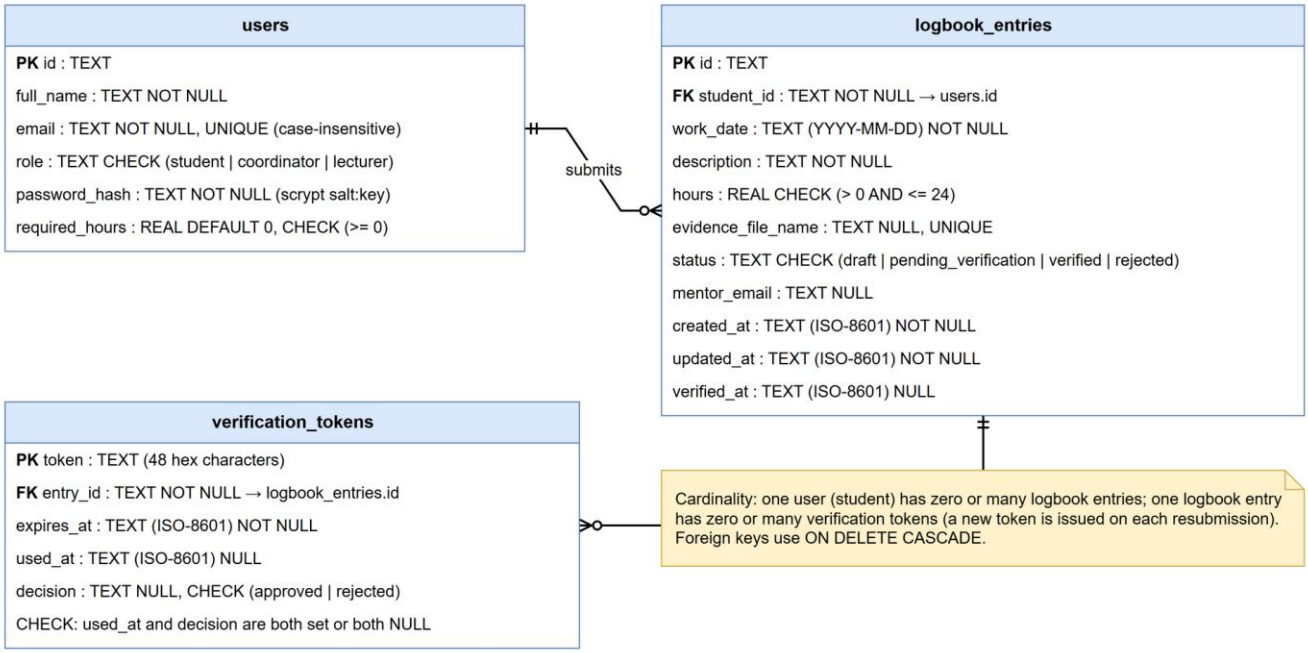


Figure 6: Entity-relationship diagram (SQLite)

Entities and keys. users stores every account (students, coordinators and lecturers), with id as the primary key and a unique, case-insensitive email. logbook_entries stores one row per day of work, with id as the primary key and student_id as a foreign key to users. verification_tokens stores one row per verification link, with the token itself as the primary key and entry_id as a foreign key to logbook_entries.

Relationships and cardinality. One student has zero or many logbook entries, and each entry belongs to exactly one student. One entry has zero or many verification tokens (a new token is created each time a rejected entry is resubmitted), and each token belongs to exactly one entry. Both foreign keys use ON DELETE CASCADE, so deleting a user cannot leave orphaned entries or tokens.

Normalisation. The design is in third normal form. Every column holds a single value (1NF); every table has a single-column primary key, so no column depends on only part of a key (2NF); and no non-key column depends on another non-key column (3NF). For example, the student's name is stored once in users and not copied into each entry. Values that can be calculated, such as a student's verified hours and their On track / Awaiting / Overdue status, are not stored at all but calculated from the entries when the dashboard is displayed, so they can never be out of date. The mentor's email is stored on the entry because the mentor belongs to the placement for that piece of work and mentors have no account of their own.

Integrity rules. CHECK constraints limit role and status to their allowed values, require hours to be more than 0 and at most 24, and require a token's used_at and decision to be set together. The application validates the same rules first and shows a friendly message; the database constraints are a second line of defence.

Design trace. The database design supports FR-04 (entry fields), FR-06 to FR-08 (tokens with expiry, use time and decision), FR-11 (status calculated from entries) and FR-12 (timestamps).

4.5 User Interface Design

The user interface was designed as a working prototype rather than paper wireframes: each screen was built in the first sprint, reviewed by the team and refined in the next sprint. The screenshots below were captured from the application running with the synthetic demonstration data.

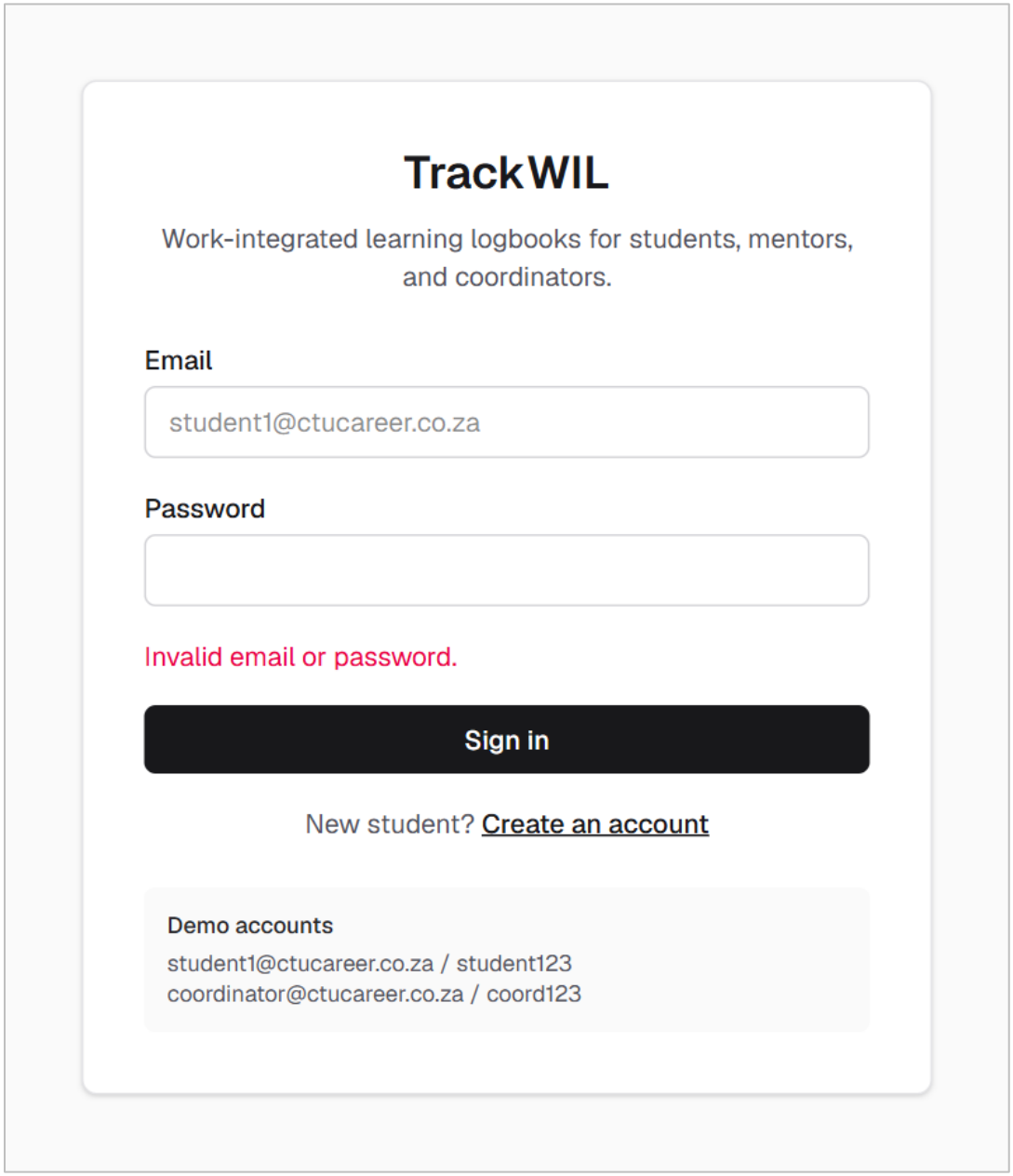


Figure 7: Sign-in page with an error for a wrong password

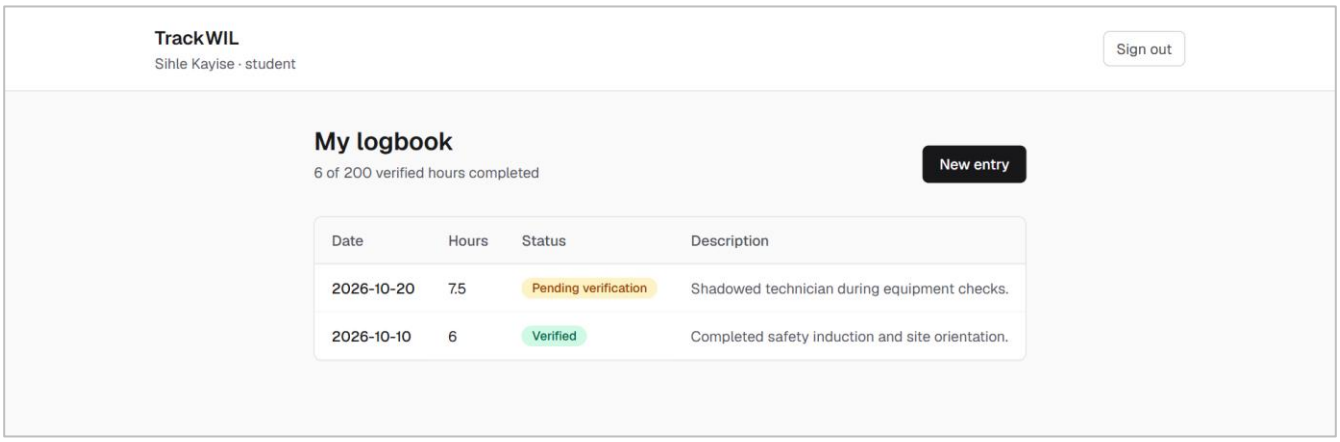


Figure 8: Student logbook showing verified hours and entry statuses

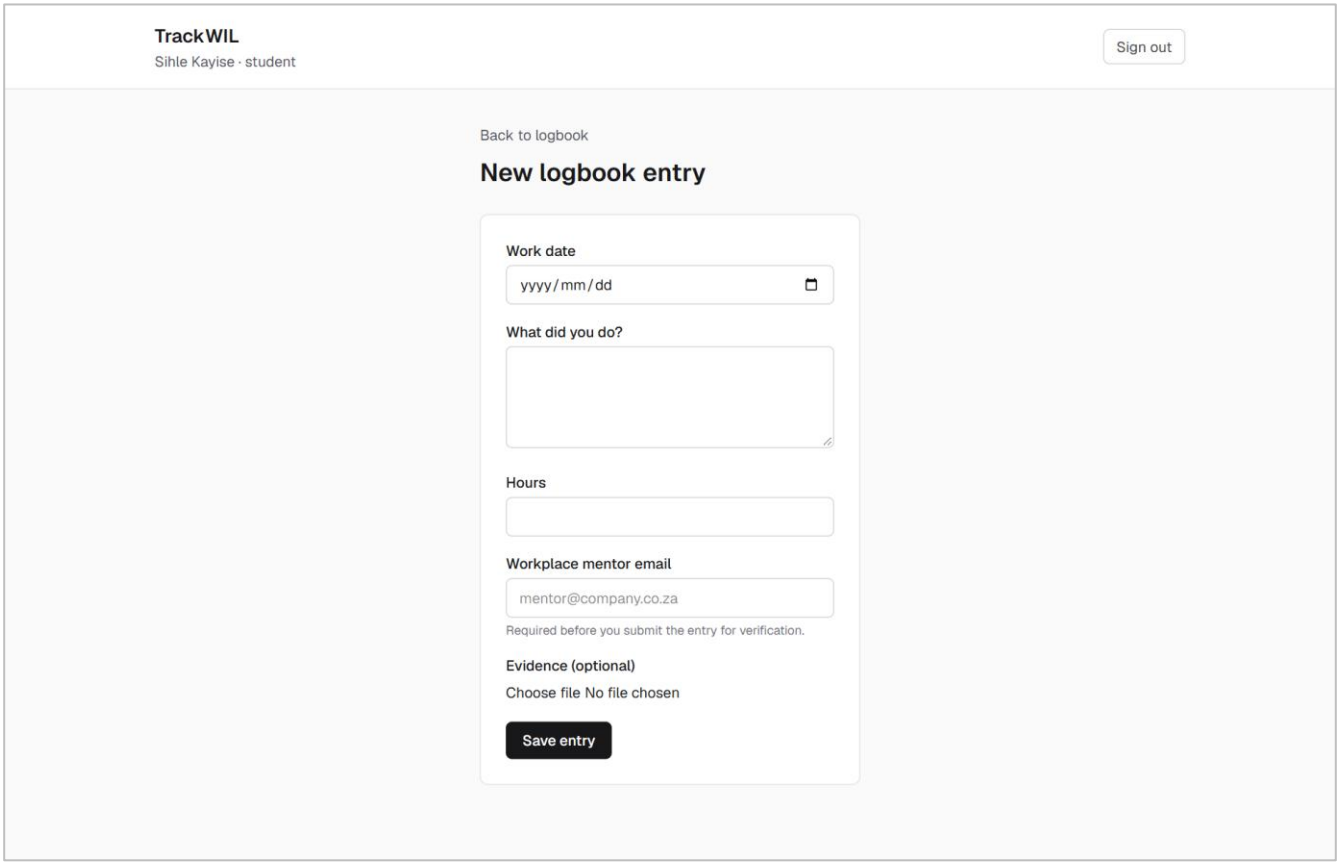


Figure 9: New logbook entry form

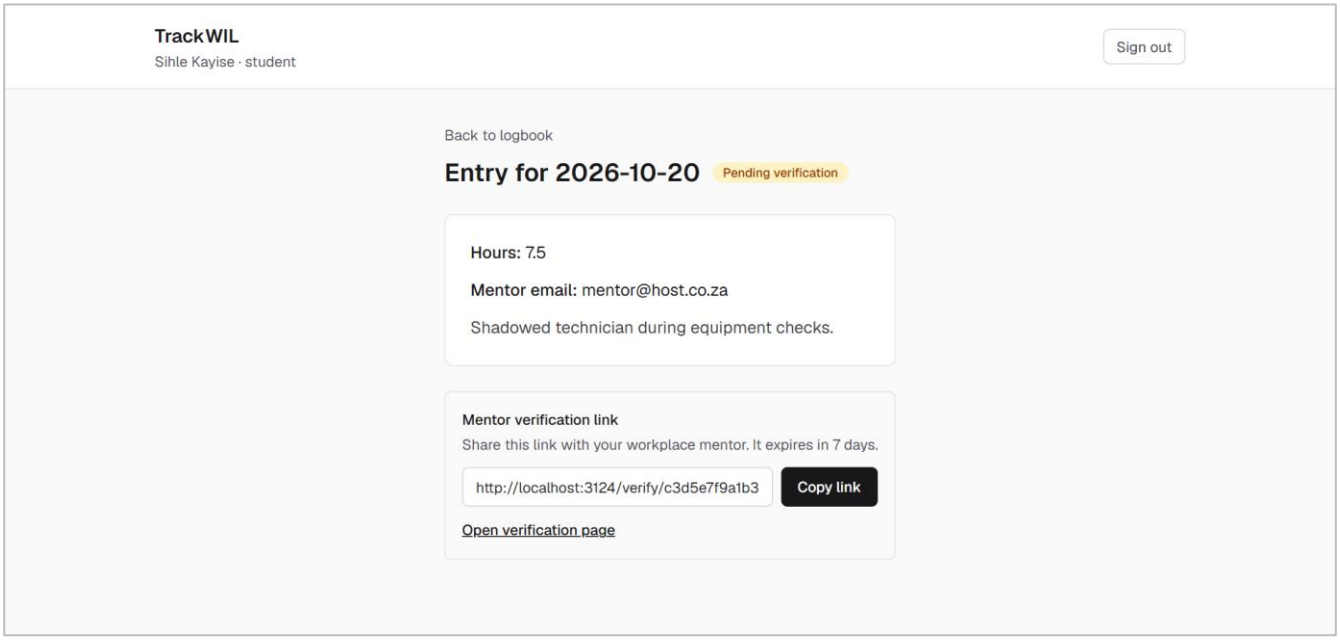


Figure 10: Entry details with the mentor verification link

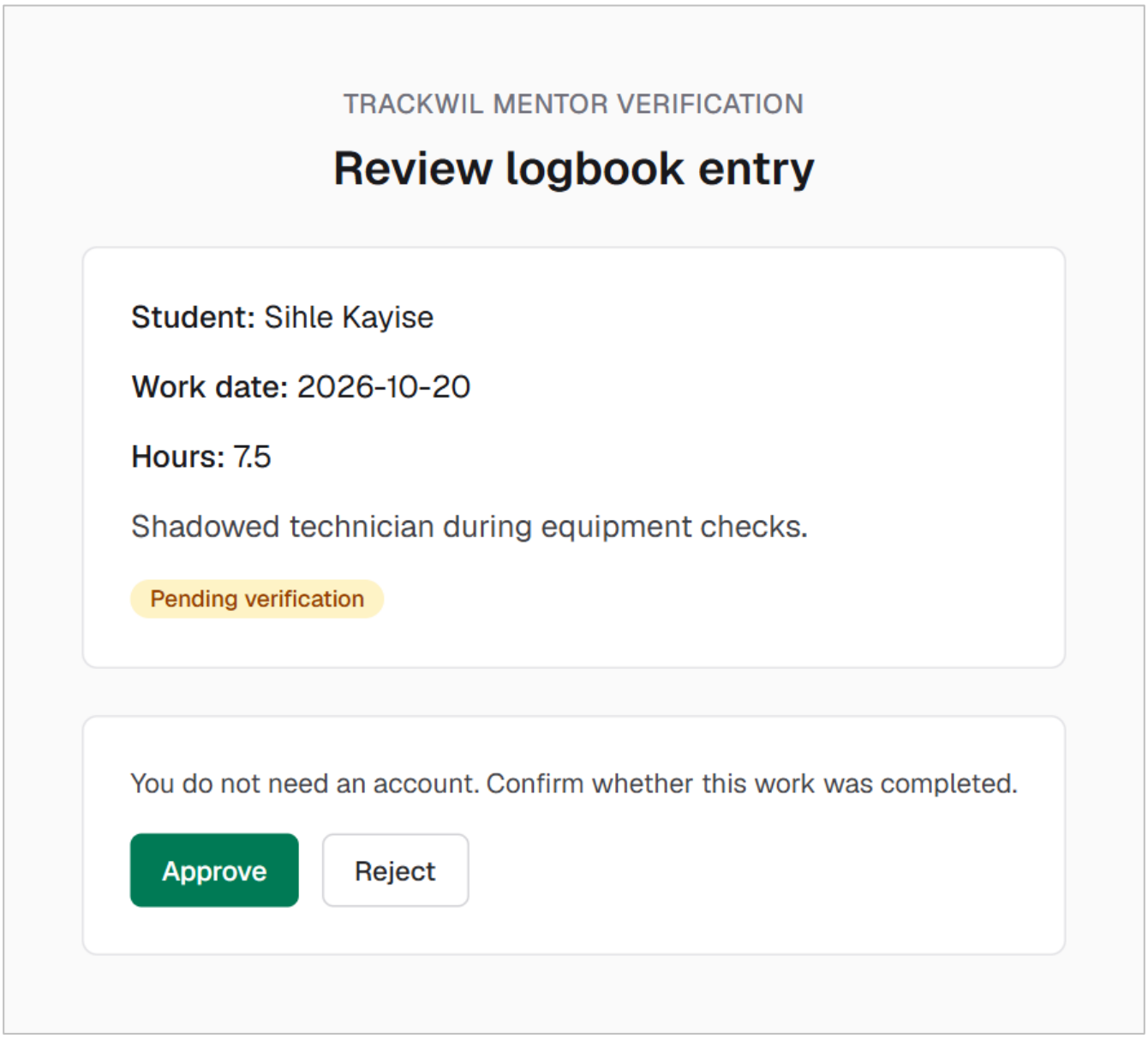


Figure 11: Mentor verification page (no sign-in required)

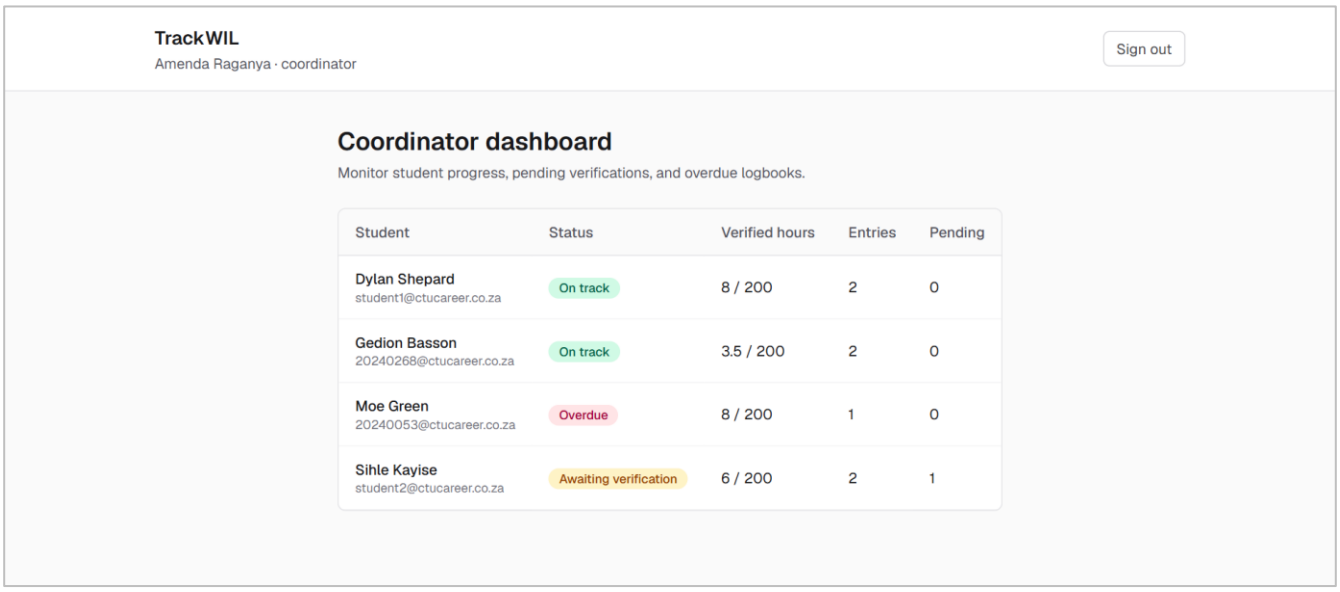


Figure 12: Coordinator dashboard with On track, Awaiting verification and Overdue statuses

Table 3: Usability decisions

Decision	Reason	Screen
Plain-language error messages next to the form (for example "Invalid email or password.")	Users can see what went wrong and correct it without leaving the page	Sign-in, sign-up, new entry
Each status shown as a coloured badge with a text label	Colour makes status quick to scan; the text means it does not rely on colour alone	Logbook, entry details, dashboard
Progress shown as "X of 200 verified hours completed" at the top of the logbook	The student's most important question is answered first	Logbook
Only four fields on the new-entry form, with evidence marked as optional and a hint under the mentor email	Keeps daily logging quick (NFR-01)	New entry
Copy link button and a note that the link expires in 7 days	The student can paste the link into an email or chat without retyping it	Entry details
Mentor page needs no sign-in and has two clear buttons	Mentors are busy and external; verification should take seconds	Mentor verification
One table on the dashboard with status first, then hours and pending count	Coordinators can see at a glance which students need follow-up	Coordinator dashboard
The header always shows the user's name, role and a Sign out button	Users always know which account they are using	All signed-in screens

4.6 Development Environment

Table 4: Development environment

Item	Tool or version
Languages	TypeScript; HTML and CSS through JSX and Tailwind; SQL (schema and queries)
Framework	Next.js 16.4 (App Router, Server Actions) with React 19.3

Item	Tool or version
Runtime	Node.js 24 (minimum 22.13 for the built-in node:sqlite module)
Database	SQLite (file data/trackwil.db), schema in database/schema.sql
Libraries	Tailwind CSS 4; Node.js crypto, fs and node:sqlite (built in)
Code quality	TypeScript compiler (type checking) and ESLint 9 with the Next.js rules
IDE	Visual Studio Code
Design tools	draw.io (UML, ERD and architecture diagrams)
Version control	Git and GitHub (https://github.com/Noobey32/TrackWIL)
Cloud services	None; the prototype runs locally
Operating system	Microsoft Windows 11

To run the application, install Node.js 22.13 or newer and run npm install and npm run dev in 05_Source_Code/trackwil, then open http://localhost:3000. The README lists the demonstration accounts. The command npm run db:reset restores the demonstration data.

4.7 Implementation

This section explains the most important modules. Short code excerpts are included only where they show how a requirement or security rule is enforced.

Project Structure

Table 5: Source code structure (05_Source_Code/trackwil)

Folder or file	Contents
database/	schema.sql (tables, keys, constraints) and seed.sql (synthetic data)
scripts/reset-db.mjs	Rebuilds the database from the SQL scripts
src/app/	Pages and routes: sign-in, signup, student, coordinator, verify, api/evidence
src/app/actions/	Server Actions: auth.ts, logbook.ts, verify.ts
src/components/	Forms, status badges, header, copy-link button
src/lib/	db.ts, repository.ts, session.ts, password.ts, signup-validation.ts, logbook-metrics.ts, types.ts
src/proxy.ts	Redirects signed-out visitors away from protected areas

Authentication and Passwords

Passwords are never stored. On sign-up the password is hashed with scrypt and a random 16-byte salt, and on sign-in the hash is recalculated and compared in constant time, as recommended by OWASP (OWASP Foundation, n.d.a). Sign-up also checks that the email is a CTU student address and that the password meets the strength rules (FR-02).

```
export function hashPassword(password: string): string {
  const salt = randomBytes(16).toString("hex");
  const derivedKey = scryptSync(password, salt, KEY_LENGTH);
  return `${salt}:${derivedKey.toString("hex")}`;
}
```

After a successful sign-in the server sets an HttpOnly session cookie and redirects the user to /student or /coordinator, depending on their role.

Role-Based Access

Access is checked on the server at two levels. The proxy redirects any visitor without a session away from /student and /coordinator. Each area's layout then loads the user and checks the role, so that a coordinator cannot open the student area and a student cannot open the dashboard. Every Server Action also checks the role and ownership again before changing data, for example:

```
const entry = await getLogbookEntryById(entryId);
if (!entry || entry.studentId !== user.id) {
  return { error: 'Entry not found.' };
}
```

Logbook Entries and Evidence Upload

createLogbookEntryAction validates the date, description and hours (more than 0 and at most 24) before saving. An evidence file is accepted only if its extension is PDF, PNG, JPG, JPEG or WEBP, and it is saved under a random UUID name so that a user can never choose or overwrite a file name on the server (OWASP Foundation, n.d.c). Files are served through /api/evidence/[fileName], which returns 401 to signed-out visitors and 403 to anyone other than the owning student, a coordinator or a lecturer (FR-13).

Mentor Verification Tokens

When a student submits an entry, the server creates a token from 24 random bytes (48 hexadecimal characters, 192 bits of randomness), which cannot practically be guessed, and sets it to expire after 7 days. The token and the new Pending verification status are saved in one transaction. When the mentor decides, the token is marked as used only if it is still unused, and the entry is updated in the same transaction:

```
const tokenUpdate = database
  .prepare(`UPDATE verification_tokens SET used_at = ?, decision = ?
    WHERE token = ? AND used_at IS NULL`)
  .run(decidedAt, decision, tokenValue);
if (tokenUpdate.changes === 0) {
  return false; // already used, e.g. a double click
}
```

Monitoring Dashboard

The dashboard calculates each student's status from their entries each time it is displayed. A student with an entry awaiting verification is shown as Awaiting verification; a student with no verified entry, or whose latest verification is more than 14 days old, is shown as Overdue; all other students are On track. Verified hours are the sum of the hours of Verified entries only.

Database Layer

db.ts opens the SQLite file, turns on foreign-key enforcement, applies schema.sql and loads seed.sql if the database is empty. It also provides an inTransaction helper that commits all statements together or rolls them all back. repository.ts contains every SQL query and returns plain TypeScript objects, so pages and actions never contain SQL.

Known Security Limitation

During the security review the team found that the session cookie stores the user's id without a signature. The cookie is HttpOnly, so it cannot be read by JavaScript on the page, but a user who edits the cookie in their own browser could change the id to impersonate another account. This defect is recorded in the Defect Log, and the planned fix is to sign the cookie (or use a random session token stored in the database) so that a modified cookie is rejected. Chapter 5 reports the retest.

4.8 Version Control

The repository on GitHub (https://github.com/Noobey32/TrackWIL) holds both the documentation and the source code. The capstone pack is organised into numbered folders from 00_Project_Administration to 12_Assessment, with the application in 05_Source_Code/trackwil, the design files and diagrams in 04_System_Design_and_Development, and each weekly submission in its own folder.

Branching and Pull Requests

The team agreed that each member creates a branch named after themselves and the task (name/task, for example moe/system-design), finishes the work there, and merges it into main through a pull request so that the others can review it. main is kept as the stable branch.

Table 6: Branches in the repository

Branch	Purpose	Merged into main
main	Stable, reviewed work	(base branch)
app-dev	First application development (Next.js scaffold and MVP)	Yes
moe/system-design	UML, ERD and architecture diagrams and the Chapter 4 draft	Yes
gedion/SRS_Documents_Chapter_3	Software Requirements Specification and Chapter 3	Pull request #2
Dylan/Revise-chapters-1-and-2	Revisions to Chapters 1 and 2 after Week 2 feedback	Yes
gedion/Final_Changes_Week_3_Submission	Week 3 submission compilation	Pull request #3
sihle-trackwil	Chapter 5, test plan and test cases	Pull request #4
gedion/sqlite-database	Replace JSON storage with the SQLite database	Pull request #5
gedion/docs-match-build	Update the documentation to match the final build	Pull request (Week 4)

Contribution by Member

Table 7: Commits and main contributions by member (at the time of writing)

Member	GitHub account	Commits	Main contributions shown in the history
Willem (Gedion) Basson	Cyber-4	20	Project proposal and topic acceptance; administration folder (00); repository structure; SRS and Chapter 3; Week 2 and Week 3 submission compilation; SQLite database migration; merging pull requests #2, #3 and #5
Moegammad Green	Noobey32 (repository owner)	16	Next.js scaffold and MVP; JSON storage and mentor verification; student sign-up validation; code documentation; draw.io diagrams; Chapter 4 draft; project charter and work plan
Dylan Shepard	FyreGH	12	Chapter 2 literature review and review matrix; reference list; formatting of Chapters 1–3; revised Chapters 1 and 2 table; Week 3 submission folder
Sihle Kayise	sihle588	4	Chapter 1 and Chapter 3 uploads; Chapter 5 draft, test plan and test cases; pull request #4

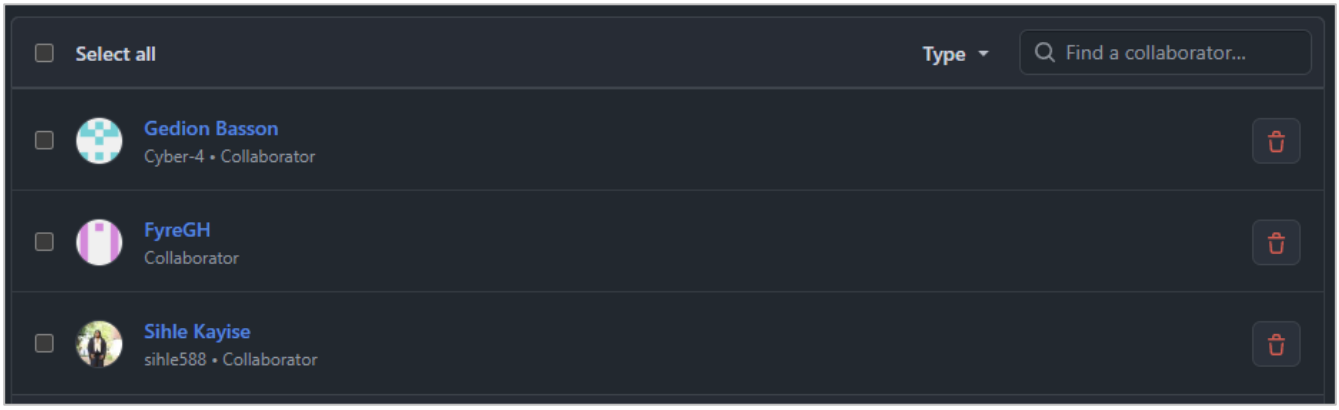


Figure 13: Collaborators on the GitHub repository

The commit history shows work on 7, 8, 9, 12, 14, 15 and 22 October 2026, spread across the project rather than in one final upload. The team recognises that the number of commits differs between members, and the Individual Contribution Forms explain each member's work in more detail, with references to specific commits and files.

4.9 Chapter Summary

This chapter presented TrackWIL's three-layer Next.js architecture, the use case, class, activity and sequence diagrams, the normalised SQLite database design and the user interface with its usability decisions. It described the development environment and the implementation of authentication, role-based access, evidence upload, one-time verification tokens, the monitoring dashboard and the database layer, and it recorded one known security limitation that will be fixed and retested. Finally, it summarised the repository structure, branches, pull requests and each member's contribution.

The core features implemented are: student sign-up and sign-in; logbook entries with optional evidence; verified hours against required hours; submission for verification with a 7-day one-time link; mentor approval or rejection without an account; and the coordinator and lecturer dashboard. Chapter 5 tests these features against the requirements.

References

OWASP Foundation (n.d.a) Password Storage Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 22 October 2026).

OWASP Foundation (n.d.c) File Upload Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html (Accessed: 9 October 2026).